

Calendar — Remaining Variants

The Last ~5%: Reasoning-Style Phrasings + a One-Page Decision Tree

Aptitude Grind

Your first three booklets (01-fundamentals, 02-fast-tricks-and-shortcuts, 03-question-pattern-catalog) cover essentially all high-frequency TCS NQT Calendar questions, and most of general placement Calendar too. This file adds the small remainder: ten reasoning-style variants that show up occasionally (more often in Infosys/Wipro-style papers than pure TCS), five extra trigger tricks, and a decision tree to pick the right method under time pressure. Everything below still reduces to the core ideas in 01-fundamentals.tex and the Anchor-Day Method in 02-fast-tricks-and-shortcuts.tex.

Category I — Reasoning-Style Variants

I1. Age + Calendar

Phrasing

“Ram was born on a Monday. His 18th birthday falls on a Wednesday. How many leap years occurred in between (assuming no 29 February birthday)?”

Method: Total weekday shift over 18 years = Wednesday – Monday = 2 days forward. Each ordinary year contributes 1 odd day, each leap year contributes 2. If L is the number of leap years, then $(18 - L)(1) + L(2) \equiv 2 \pmod{7} \Rightarrow 18 + L \equiv 2 \pmod{7} \Rightarrow L \equiv 5 \pmod{7}$. Since L must be a small number of leap years in an 18-year span (4 or 5 typically), **L = 5**.

I2. Date Mirror Questions

Phrasing

“If 12 March is a Tuesday, what day is 21 March?”

Method: Never recount from the start of the month — just take the difference in dates and reduce mod 7. $21 - 12 = 9$ days; $9 \bmod 7 = 2$; Tuesday + 2 = **Thursday**. This is Trick 7 below, applied directly.

I3. Consecutive Leap-Year Logic

Phrasing

“Three leap years occur between 1 January of year A and 1 January of year B (A earlier than B). If 1 January of year A is a Sunday, what day is 1 January of year B, given there are 10 years between A and B?”

Method: Same odd-day accumulation as I1, just phrased as a given leap-year count instead of an asked one. Odd days = $(10 - 3)(1) + 3(2) = 7 + 6 = 13 \equiv 6 \pmod{7}$; Sunday + 6 = **Saturday**. Always check whether the leap-year count is *given* (as here) or *asked* (as in I1) — the same equation is just solved for a different variable.

I4. Financial / Office Calendar (Working-Day Count)

Phrasing

“An office is closed on all Sundays and on the second Saturday of every month. How many working days are there in a 30-day month that starts on a Saturday?”

Method: List the calendar out mod 7 for the Sundays and identify which Saturday is the “second” one; subtract both sets from the total days, watching for any date that is *both* a Sunday and a second-Saturday-adjacent date. With the month starting on Saturday: Saturdays fall on 1, 8, 15, 22, 29 (2nd Saturday = 8th); Sundays fall on 2, 9, 16, 23, 30 (5 Sundays). Total off-days = 5 (Sundays) + 1 (second Saturday) = 6; **working days = 30 - 6 = 24**.

I5. Number of Sundays/Mondays Between Two Dates

Phrasing

“How many Sundays are there between 15 January and 15 April (inclusive of the range) of the same year, given 15 January is a Thursday?”

Method: Find the day-of-week of the range’s start, locate the first Sunday, then count forward in steps of 7 to the end. 15 Jan is Thursday $\Rightarrow$ first Sunday is 18 Jan. Days from 18 Jan to 15 Apr = 13 (rest of Jan) + 28 (Feb) + 31 (Mar) + 15 (Apr) = 87 days $\Rightarrow 87/7 = 12$ full weeks plus a remainder, giving **13 Sundays**. Always find the *first* occurrence of the target weekday in the range before dividing by 7 — don’t divide the whole range by 7 directly.

I6. First/Last Working Day

Phrasing

“Find the first Monday after 15 June, given 15 June is a Friday.”

Method: Find the weekday gap to the target day, then add it as a date offset. Friday to Monday is 3 days forward $\Rightarrow$ **18 June**. For a “last weekday before a date” version, the same idea works backward: find the gap going back to the most recent occurrence of the target weekday.

I7. Month Identification from Weekday Counts

Phrasing

“A particular month has 5 Sundays, 5 Mondays, and 5 Tuesdays. Which day of the week does the 1st of that month fall on, and how many days does the month have?”

Method: A weekday occurs 5 times in a month only if the month has at least 29 days *and* the month’s first few days include that weekday enough times. For three consecutive weekdays (Sun, Mon, Tue) to each occur 5 times, the month must have 31 days, and the 1st must fall on the *first* of the three repeated weekdays. **The 1st is a Sunday**, and the month has **31 days**. (See Trick 10 below for why “days – 28” is the fast way to check this.)

18. Missing Calendar (Given One Fact, Reconstruct the Month)

Phrasing

“In a certain month, 1 January is a Friday. What day of the week is 1 February in a non-leap year?”

Method: January has 31 days $\Rightarrow 31 \bmod 7 = 3$ odd days $\Rightarrow$ Friday + 3 = **Monday**. This is the same relative-arithmetic idea as `02-fast-tricks-and-shortcuts.tex`, just applied to “reconstruct the next month’s start” instead of a specific date within the same month.

19. Calendar Table / Grid Interpretation

Phrasing

“You are shown a calendar grid for a given month where 1st falls on a Wednesday. What date is the third Saturday of that month?”

Method: Same as the Nth-weekday method in `03-question-pattern-catalog.tex` Category C — find the first occurrence of the target weekday, then add 7 for each subsequent occurrence. 1st is Wednesday $\Rightarrow$ first Saturday is the 4th $\Rightarrow$ third Saturday = $4 + 7 + 7 = 18$. **18th**. When the question shows an actual calendar image rather than describing it, the method is identical — just read the first occurrence directly off the grid instead of computing it.

I10. Date Validation

Phrasing

“Which of these dates cannot exist: 31 April, 30 February, 29 February 2023, 31 June?”

Method: Check each date against the fixed days-per-month rule (30 days have September, April, June, and November) and the leap-year rule for 29 February. **31 April, 30 February, 29 February 2023, and 31 June are all invalid** — April, June, September, and November never have a 31st, no month has a 30th of February, and 2023 is not a leap year so it has no 29 February. This is an easy, almost purely-recall question type — don’t overthink it.

Extra Trigger-Word Tricks

Trick 6 — Only Weekdays Are Mentioned (No Dates)

If a question never gives an actual date, just a weekday and a day-count, skip the full Anchor-Day Method entirely: take the day count mod 7 and shift directly from the given weekday.

Trick 7 — Same Month Two Dates

If both dates fall in the same month, never recompute the calendar from the 1st. Just take the difference between the two date numbers and reduce it mod 7 (used in I2).

Trick 8 — Same Date Different Year

When comparing the same date across two different years, immediately check whether 29 February falls in between. If it does, the shift is 2 odd days per year crossed that includes a leap day; otherwise it's 1 odd day per year.

Trick 9 — Nth Weekday of a Month

Never try to multiply directly to the Nth occurrence. Always find the *first* occurrence of the target weekday, then add 7 for each additional occurrence needed (used in I6, I7, I9).

Trick 10 — “Which Weekdays Occur 5 Times”

Don't draw out the whole calendar. A month has (days – 28) “extra” days beyond four full weeks, and exactly that many *consecutive* weekdays (starting from the 1st) occur 5 times. A 31-day month has 3 such weekdays; a 30-day month has 2; a 28-day month (February, non-leap) has none; a 29-day month (leap February) has exactly 1.

Decision Tree — Which Method to Use

Use this under time pressure to skip straight to the right method instead of considering multiple approaches:

Decision Tree

- **Only weekdays, no dates?** → $N \bmod 7$ (Trick 6)
- **Two dates in the same month?** → Difference mod 7 (Trick 7)
- **A specific date, asks for the day it falls on?** → Anchor-Day Method (§1 of Booklet 02)
- **Leap year status affecting a year-to-year shift?** → Check if 29 Feb lies between (Trick 8)
- **A month name / month code directly?** → Month Code Method (§2 of Booklet 02)

- **The Nth or last occurrence of a weekday?** → First occurrence, then add multiples of 7 (Trick 9)
- **Whether a weekday occurs 5 times in a month?** → days – 28 (Trick 10)
- **Whether two years share the same calendar?** → Running total of odd days (Catalog, Category D)
- **An age, birthday, or leap-year count as the unknown?** → Odd-day accumulation equation (I1, I3)

What This File Deliberately Skips

Consistent with the Time & Work module’s philosophy: this file does not cover exotic multi-calendar conversions (Islamic/Hindu calendar mapping), astronomical leap-second edge cases, or Gregorian vs. Julian calendar transition-date puzzles. These essentially never appear in TCS NQT or general placement-level Calendar questions.

Updated Priority Order

With this file added, the full priority order across all four booklets becomes:

1. **A** — Direct day lookup (base method — Anchor-Day)
2. **B** — Relative day arithmetic (no calendar needed)
3. **I2** — Date mirror questions (same family as B)
4. **C** — Nth / last weekday of a month
5. **I6/I9** — First/last working day, calendar table interpretation (same family as C)
6. **D** — Same calendar / repeating year
7. **E** — Leap year reasoning
8. **I1/I3** — Age + calendar, consecutive leap-year logic (same family as E)
9. **F** — Date-to-date and span problems
10. **I5** — Number of Sundays/Mondays between two dates (same family as F)
11. **G** — Reverse problems
12. **I8** — Missing calendar / reconstruct the month (same family as G)
13. **H** — Logic puzzles and word problems
14. **I7** — Month identification from weekday counts (same family as H)
15. **I4** — Financial/office calendar working-day count
16. **I10** — Date validation (lowest effort, easy marks — don’t skip it)

At this point Calendar coverage is effectively complete for TCS NQT and most general placement tests. The highest-value next step is the same as with Time & Work: solve 40–60 mixed Calendar questions under a timer until each phrasing triggers the right branch of the decision tree instantly, rather than continuing to add more theory.